



COS 132 - Imperative Programming

Study Unit 5: Control Structures - Selection Statements

Copyright ©2024 by Linda Marshall, Inge Odendaal and Mark Botros. All rights reserved.

Contents

5.1	Introduction	2
5.2	Ternary If-Statements	2
5.2.1	Simple Ternary If-Statements	3
5.2.2	Nested Ternary If-Statements	4
5.3	If-Statements	5
5.3.1	Simple If-Statements	5
5.3.2	Multi-way If-Statements	8
5.3.2.1	Nested If-Statements	8
5.3.2.2	Chained Else If-Statements	9
5.4	Switch-Statements	10
5.5	Factorising and Translating Selection Statements	12
5.5.1	Factorising of If-Statements	13
5.5.1.1	Factorisation of Statements	13
5.5.1.2	Condition swapping	15
5.5.2	Translating Between If and Switch	16
5.5.2.1	From If to Switch	16
5.5.2.2	From Switch to If	17
5.5.2.3	Considerations for Translations between Selection Statements	18
5.6	Testing and Debugging	18
5.6.1	Print Statements	19
5.6.2	Assertion	19
5.6.3	Using Assertions to Enforce Pre- and Post-conditions	19
5.7	Document Change Log	20

5.1 Introduction

The selection statement allows specific statements to be executed depending on the outcome of a condition. This results in branching in the code and not all statements are executed each time the code executes. The result of applying a condition will evaluate to either *true* or *false*.

The general principle of a selection statement is that specific statements will execute if the condition is *true*, otherwise they will not execute. Additional structures may be defined to manage the case where the condition is *false* to remove the need to test the negation of the condition (with the not (!) operator). Refer to Study Unit 3, Section 3.4.4 for a recap on conditions (also referred to as boolean expressions).

There are two types of selection statements in C++, the if-statement and the switch-statement. A third type of statement exists, the conditional statement which translates directly into an if-statement. Thus study unit will begin by discussing the ternary-if before considering the traditional if- and switch-statements.

5.2 Ternary If-Statements

The ternary-if, makes use of the ternary operator (? :), also known as the **conditional operator**. The operator provides a concise way to express a simple conditional statement in C++. It takes three operands: a condition, an expression for true, and an expression for false. The syntax is as follows:

```
<condition> ? <expression_true> : <expression_false>;
```

Listing 5.1: Ternary If Syntax

- **<condition>:** Must be a boolean expression
 - Direct boolean values (**true**, **false**)
 - Comparisons (**a == b**, **a < b**, **a >= b**, etc.)
 - Logical operations (**a && b**, **a || b**, **!a**, etc.)
- **<expression_true> and <expression_false>:** Any expressions that results in a value
 - Literals (**42**, **3.14**, **'c'**, etc.)
 - Variables (**myVar**, **result**, etc.)
 - Function calls (**calculate()**, **getMax(a, b)**, etc.)
 - Arithmetic expressions (**a + b**, **a * b**, etc.)
- Both the **<expression_true>** and **<expression_false>** need to be of the same type, or convertible to the same type.

If the **<condition>** is true, the operator evaluates and returns the result of evaluating **<expression_true>**; otherwise, it returns the resultant value after the evaluation of **<expression_false>** has taken place.

5.2.1 Simple Ternary If-Statements

In C++ the ternary operator's structure, maps onto a simple if-statement and is therefore referred to as a ternary-if. The ternary-if can be used to perform certain actions, or to assign/return a result based on a condition.

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two numbers:\n";
    cin >> a >> b;
    int max = (a > b) ? a : b;
    cout << "The greater number is " << max << endl;
    return 0;
}
```

Listing 5.2: Using the Ternary Operator in C++

In this example, the user is prompted to enter two numbers. The ternary operator is used to determine the greater of the two numbers and assign it to the variable **max**.

In C++, the ternary operator can also use function calls as the expressions for both the true and false cases.

```
#include <iostream>
using namespace std;

int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

int main() {
    int num1, num2;
    char operation;
    cout << "Enter two numbers: ";
    cin >> num1 >> num2;
    cout << "Choose operation (a for add, s for subtract): ";
    cin >> operation;

    bool condition = (operation == 'a');

    int result = condition ?
        add(num1, num2) : subtract(num1, num2);

    cout << "The result is: " << result << endl;
}
```

```

    return 0;
}

```

Listing 5.3: Ternary Operator with Function Calls

In this example, Listing 5.3, the user enters two numbers followed by their choice of operation ('a' for addition and any other character for subtraction). The ternary operator evaluates the condition to determine whether the character entered by the user is 'a'. If it is, the 'add' function is called; otherwise, the 'subtract' function is called by default. The result of the chosen operation is then displayed.

Question 5.1

What output is produced by the ternary-if in Listing 5.3 when the input values are 10 for a and 20 for b?

Question 5.2

Write a C++ program using a ternary-if that takes two numbers as input, performs division, and prints the result. If the denominator is 0, the program should print '1' as the result instead of performing the division.

Question 5.3

Write a C++ program using a ternary operator to check whether a given integer is odd or even. Print "Odd" for odd numbers and "Even" for even numbers.

5.2.2 Nested Ternary If-Statements

The ternary operator can be nested IN C++, This means that the expression to be evaluated can itself be another ternary if-statement.

```

#include <iostream>
using namespace std;

int main() {
    int score;
    cout << "Enter_your_score:_";
    cin >> score;

    //Simple nesting
    string grade = (score < 50 ) ? "Fail" :
                   (score < 75) ? "Pass" : "Distinction";

    cout << "Your_grade_is:_ " << grade << endl;

    //Complex nesting
    string detailed = (score >= 90)? "High_Distinction" :
                      (score >= 75) ? "Distinction" :
                      (score >= 60) ? "Credit" :
                      (score >= 50) ? "Pass" : "Fail";

    cout << "Your_detailed_grade_is:_ " << detailed << endl;
}

```

```

    return 0;
}

```

Listing 5.4: Nested Ternary Operator

Ternary operators can be nested arbitrarily many times (chained), as long as in the end each expression (and sub-expression) evaluates to the same (or implicitly convertible) types. While this is a concise way of writing the logic, it might be challenging to read and maintain. It's best used for simple conditions and when clarity is not compromised.

The ternary operator is very effective for assigning values based on a condition or for simple condition checks within a statement.

Question 5.4

What output is produced by Listing 5.4 when the user inputs 74.

Question 5.5

Given three integer values 'a', 'b', and 'c', write a C++ program using the ternary operator to find the smallest value and print it.

Question 5.6

Write a C++ function with signature `string classifyAge(int age)` using ternary operators that takes an age as input and returns whether the person is classified as "Child", "Adult", or "Senior". The age categories are defined as follows: Child (below 18), Adult (18 to 64), Senior (65 and above).

5.3 If-Statements

An if-statement evaluates a condition. Depending on the outcome of the condition, either `true` or `false`, a single statement or block of statements are executed.

5.3.1 Simple If-Statements

The basic syntax of the if-statement is as follows:

C++ Single Statement	C++ Block of Statements
if (<condition>) <statement>;	if (<condition>) { <block>; }
if (<condition>) <true_statement>; else <false_statement>;	if (<condition>) { <true_block>; } else { <false_block>; }

The simple if-statement translates directly to the ternary if-statement that includes both a `<true_statement>` and a `<false_statement>`. That is the high-level code of:

```

if (<condition>)
    <true_statement>;
else
    <false_statement>;

```

translates to the ternary if-statement of:

```
<condition> ? <true_statement> : <false_statement>;
```

Consider the following C++ program illustrating the use of a single statement for the result of the condition.

```

#include <iostream>
using namespace std;

int main() {
    int x;
    cout << "Type_in_an_integer_value:_";
    cin >> x;
    if (x < 10)
        cout << "The_value_is_<_10" << endl;
    else
        cout << "The_value_is_>=_10" << endl;

    return 0;
}

```

Listing 5.5: Basic If-Statement in C++

In this C++ program, the value of `x` is compared to 10, and a message is displayed based on whether `x` is less than 10 or greater than or equal to 10.

The following C++ program uses blocks of code (encapsulated in the curly brackets). This C++ program takes two integers from the user, compares them, and either adds them or subtracts them based on their values. It then displays the results.

```

#include <iostream>
using namespace std;

int main() {
    int x, y;
    cout << "Type_in_an_integer_value:_";
    cin >> x;
    cout << "Type_in_another_integer_value:_";
    cin >> y;

    if (x < y) {
        cout << x << "_is_less_than_" << y << endl;
        cout << x << "_plus_" << y << "_=" << x + y << endl;
    } else {
        cout << x << "_is_not_less_than_" << y << endl;
        cout << x << "_minus_" << y << "_=" << x - y << endl;
    }
}

```

```

    }
    return 0;
}

```

Listing 5.6: If-Else with Blocks in C++

It is possible to mix single statement and the block of statements in a single if-statement. A sound programming principle to follow is to only use blocks when coding your if-statements.

Question 5.7

What is displayed on the screen when Listing 5.6 is compiled and run with values 14 for `x` and 29 for `y`?

Note that the if-statement of program Listing 5.6 could also be rewritten in the following way using two different functions (provided they are appropriately defined with the correct statements).

```

if (x < y) {
    printLessThanMessage();
} else {
    printGreaterEqualMessage();
}

```

Listing 5.7: If-Else with Blocks in C++

This approach is better where your programs get longer. This embodies the principle of divide and conquer!

Question 5.8

Are the following two program segments equivalent? That is, is the outcome of the program segments the same for all values of `x`?

```

#include <iostream>
using namespace std;

int firstFunction(int x){
    if (x > 100) {
        cout << x << endl;
        x += 2;
    } else {
        x += 2;
    }

    return x;
}

int secondFunction(int x){
    if (x > 100) {
        cout << x << endl;
    }
}

```

```

        x += 2;

    return x;
}

int main() {
    int x;
    cout << "Enter a value: ";
    cin >> x;
    firstFunction(x);
    // versus
    secondFunction(x);

    return 0;
}

```

Listing 5.8: Program Segment Equivalence

5.3.2 Multi-way If-Statements

If-statements in C++ can be structured in various ways to achieve multi-way branching. One common approach is to use a series of if-else statements, which can be either nested or sequenced using else if. Both approaches can achieve the same outcome, but they differ in readability and structure.

Consider categorizing the ages of individuals into three categories:

- young (0-<35),
- middle-aged (35-<65), and
- ancient (65+),

and printing the results to the screen.

5.3.2.1 Nested If-Statements

Nested if-statements occur when an if or else if block contains another if-statement. This nesting can continue to any level of depth as needed.

```

#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Type in an age [0 to 120]: ";
    cin >> age;

    if (age < 35) {

```



```

        cout << "You_are_young" << endl;
    } else {
        if (age < 65) { // age >= 35 and age < 65
            cout << "You_are_middle-aged" << endl;
        } else {
            cout << "You_are_ancient" << endl;
        }
    }
    return 0;
}

```

Listing 5.9: True Nested If-Statements

This structure shows the hierarchy of conditions, where the subsequent if-statements are considered only when the previous conditions are not met.

5.3.2.2 Chained Else If-Statements

Alternatively, else if chains can be used to simplify the nested if structure. The else if approach is more readable, especially when dealing with multiple levels of conditions.

```

#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Type_in_an_age_[0_to_120]:_";
    cin >> age;

    if (age < 35) {
        cout << "You_are_young" << endl;
    } else if (age < 65) { // age >= 35 and age < 65
        cout << "You_are_middle-aged" << endl;
    } else { // age >= 65
        cout << "You_are_ancient" << endl;
    }
    return 0;
}

```

Listing 5.10: Chained Else If-Statements

This approach makes the code easier to follow without the additional indentation that comes with nested if-statements.

Both of these approaches categorise individuals into age groups. While syntactically different, they are functionally equivalent.

Question 5.9

Write a C++ program that asks the user to enter any two numbers. The two numbers, as well as the highest of these numbers, must be printed. Assign the highest value to a separate variable, e.g., **highest**. Use a function to do this.

Question 5.10

Write a C++ program that asks the user to enter any three numbers. All three numbers, as well as the highest of these numbers, must be printed. Use nested if-statements. If you try to do this without using the same function as in the previous question, you might find it quite difficult to solve.

Question 5.11

Write a C++ program that asks the user to enter two numbers between 1 and 20, `num1` and `num2` respectively. `num1` must be compared to `num2`. If `num1` is lower, exactly the same, or higher, respectively, an appropriate message must be printed. Use chained else-if statements.

Question 5.12

Write a C++ program that reads in a T-shirt size ('S', 'M', 'L', 'X') and displays the T-shirt size in words (e.g., 'Small', 'Medium', 'Large', or 'Extra Large'). Use nested/chained if-statements.

Question 5.13

Write a boolean function that determines whether a year is a leap year or not. The function should have one input parameter, the year, and return true if the year is a leap year, and false if it is not.

(Note: All years divisible by 4 are leap years. However, century years are an exception - they are only leap years when they are divisible by 400. For example, the year 1900 was not a leap year, but the year 2000 was one. Hint: Use the modulus operator (%) to test for divisibility.)

5.4 Switch-Statements

The switch-statement allows for multiple choices to be made based on the value of a variable. This is handy to use instead of multiple nested if-statements, or chained else-if statements. The switch-statement is useful when you have a variable that can be one of several discrete values, and the action that you need to perform if based on the value.

The syntax of a switch-statement in C++ is as follows:

```
switch (expression) {
    case constant1:
        statement(s);
        break;
    case constant2:
        statement(s);
        break;
    ...
    default:
        statement(s);
}
```

Listing 5.11: Syntax of Switch-Statement

Here, `expression` is evaluated and compared against the constants. If a match is found, the corresponding block of statements is executed. The `break` statement prevents the execution from falling through to the next case. Falling through means that all `statement(s)`

following the first match will be executed, even those that form part of a separate case. This is sometimes desirable, and thus the **break** statement is optional. The **default** case is optional and is executed if none of the constants match the expression.

Let's use a switch-statement to handle the different T-shirt sizes entered by a user, instead of using chained else-if (Hint, refer to Question 5.12).

```
#include <iostream>
using namespace std;

int main() {
    char tShirtSize;
    cout << "Type in a T-Shirt size (S, M, L, or X): ";
    cin >> tShirtSize;

    switch (tShirtSize) {
        case 'S':
            cout << "Small";
            break;
        case 'M':
            cout << "Medium";
            break;
        case 'L':
            cout << "Large";
            break;
        case 'X':
            cout << "Extra Large";
            break;
        default:
            cout << "Incorrect T-Shirt size!";
    }
    cout << endl;
    return 0;
}
```

Listing 5.12: Basic Switch Example

Question 5.14

What will the output of Listing 5.12 be if the user enters 'S'? What about 'B'?

Question 5.15

What would happen when all the **break** statements are removed? Write the output when the user enters 'S'. What about 'B'?

If you want to perform certain actions if the expression is one of a range of values. This can be done by grouping case labels that execute the same code. In this case, we are effectively letting it "fall through" and only breaking after the actions for the range is performed.

```
#include <iostream>
using namespace std;
```

```

int main() {
    int tShirtSize;
    cout << "Type_in_a_T-Shirt_size_(8-24):_";
    cin >> tShirtSize;

    switch (tShirtSize) {
        case 8: case 9: case 10:
            cout << "Small";
            break;
        case 11: case 12: case 13: case 14:
            cout << "Medium";
            break;
        case 15: case 16: case 17: case 18:
            cout << "Large";
            break;
        case 19: case 20: case 21: case 22: case 23: case 24:
            cout << "Extra_Large";
            break;
        default:
            cout << "Incorrect_T-Shirt_size!";
    }
    cout << endl;
    return 0;
}

```

Listing 5.13: Handling Ranges in Switch-Statements

Question 5.16

Write a program that determines the tax payable (15% VAT) on a T-shirt price as well as the final amount. Small, medium, large, and extra-large T-shirts cost R19, R29, R39, and R59 respectively (before VAT). The program must ask for the T-shirt size and work out the tax according to the size and price of the T-shirt. Use separate functions for reading in the T-Shirt size and calculating the tax and final price. Provide user-friendly output, also showing the T-shirt size in words. Format the values of the prices to two decimals.

Question 5.17

What would happen if 's' is entered when Listing 5.12 is compiled and run? Can you think of way(s) to fix the problem?

5.5 Factorising and Translating Selection Statements

There are times when if-statements have changed over time to a point where some code is duplicated or becomes redundant when writing if-statements. It is these cases the is becomes necessary to factorise this redundancy out of the statement and present cleaner and easier to read code. When changing code from one form to another, the notion of code equivalence is important to keep in mind. A discussion of how to go about factorising if-statements to equivalent if-statements to improve the code and remove redundancy is discussed in Section 5.5.1.

As both if-statements and switch-statements deal with selection. It is conceivable that there are if-statements that can be written in terms of switch-statements and vice-versa. Section 5.5.2, discusses this translation between if-statements and switch-statements.

5.5.1 Factorising of If-Statements

Factorising of if-statements can be done to *simplify* the code written. There are a number of factorisation techniques. These techniques fall into two broad categories, factorisation of statements and factorisation of conditions.

5.5.1.1 Factorisation of Statements

Factorisation of statements removes the issue of code redundancy from the code-base. That is, repeated code that will be executed irrespective of the outcome of the evaluation of the condition will be consolidated and specified only once. This will increase the readability, understandability and maintainability of the code.

Three types of factorisation can be applied to if-statements. These are bottom-factorisation, top-factorisation and unnecessary factorisation. Each will be illustrated in the sections below.

Bottom-factorisation. Moves the statements that are duplicated at the end of the true and false blocks of an if-statement to after the if-statement.

Consider the high-level code snippet given in the column on the left of the table below. `<statement3>` is executed irrespective of what the result of the condition is. `<statement3>` can therefore be factored out and placed below the if-statement (refer to the right-hand column). Functionally the code on the left is equivalent to the code on the right in the table.

<pre> if (<condition>) { <statement1>; <statement3>; } else { <statement2>; <statement3>; } </pre>	<pre> if (<condition>) { <statement1>; } else { <statement2>; } <statement3>; </pre>
--	--

An example of bottom-factorisation is given in Question 5.8.

Top-factorisation. Top-factorisation is similar to bottom-factorisation. In this case the common code is the first statement(s) that is/are executed in both the true and false blocks. Unlike with bottom-factorisation, the statements can only be factored to before the beginning of the if-statement if the statement(s) do not influence the condition of the if-statement.

For the high-level code snippet given in the left column of the table below, `<statement1>` may only be removed resulting in the code given in column 2, if `<condition>` is not influenced by the statement.

<pre> if (<condition>) { <statement1>; <statement2>; } else { <statement1>; <statement3>; } </pre>	<pre> <statement1>; if (<condition>) { <statement2>; } else { <statement3>; } </pre>
--	--

Consider the following example.

```

int myFunction(int x) {
    if (x < 0) {
        x++;
        return 10 * x;
    } else {
        x++;
        return 10 / x;
    }
}

```

Listing 5.14: Incorrect useage of Top-factorisation Influencing the Condition

In this example, the application of top-factorisation will not result in equivalent code in the line incrementing `x` is placed before the if-statement.

Question 5.18

Trace the program given in Listing 5.14 before factorisation and after factorisation. Where does the equivalence of the code break?

Unnecessary-factorisation. Unnecessary-factorisation is concerned with the factorisation of the condition. Testing the condition takes time, unnecessary testing of the same condition slows down the execution of the program. Consider the if-statement on the left is in the table below. If the condition for the first if is the same as the condition for the else-if, and `<statement1>` has not changed any variable or value that will impact the outcome of the condition tested by the else-if, then it is not necessary to test the condition a second time. That is, `<statement2>` may be executed immediately after `<statement1>` without retesting the condition.

<pre> if (<condition>) { <statement1>; } else if (<condition>) { <statement2>; } else { <statement3>; } </pre>	<pre> if (<condition>) { <statement1>; <statement2>; } else { <statement3>; } </pre>
--	--

Consider the following code snippet:

```

if (x < 10) {
    cout << "The_value_of_x_is:_ " << x << endl;
}

```

```

    if (x < 10) {
        x += 10;
    } else {
        x -= 10;
    }
}

```

Listing 5.15: Unnecessary factorisation

The statement, `cout << "The_value_of_s_is:" << x << endl;` does not impact on the condition and therefore the first else-if statement may be removed, resulting in the following equivalent code.

```

if (x < 10) {
    cout << "The_value_of_s_is:" << x << endl;
    x += 10;
} else {
    x -= 10;
}

```

Listing 5.16: Unnecessary factorisation Applied

5.5.1.2 Condition swapping

Condition swapping¹ means that instead of testing for the condition to be true, you test for the complement (i.e., the opposite) of the condition. For example, if the condition were to be `(x > 10)` the complement (!) of the condition would be:
`(x < 10)` or `!(x > 10)`.

Consider the table below. When the complement of the the condition is applied to the if-statement on the left, the statements in the true block and the false block of the if-statement need to be switched. The right-hand side of the table is the result of applying this switch.

<pre> if (<condition>) { <statement1>; } else { <statement2>; } </pre>	<pre> if !(<condition>) { <statement2>; } else { <statement1>; } </pre>
--	---

Assume there is no true block for the condition, only a false block to be coded. Testing the complement of the condition will result in more concise code. Refer to the table below for a high-level illustration of how condition swapping impacts the high-level code.

<pre> if (<condition>) { } else { <statement>; } </pre>	<pre> if !(<condition>) { <statement>; } </pre>
---	---

¹mainly UK version for “swapping” in the US. The terms will be used interchangeably in the study units.

Consider the following implementation of the `lessThan` function.

```
bool lessThan(int a, int b) {
    if (a < b) {
        return a;
    } else {
        return b;
    }
}
```

Listing 5.17: `lessThan` function

The complement of the condition results in the following code:

```
bool lessThanComplement(int a, int b) {
    if (a >= b) {
        return b;
    } else {
        return a;
    }
}
```

Listing 5.18: Complement of `lessThan` function

Question 5.19

Write a main function to test whether the outcome of calling `lessThan` is equivalent to calling `lessThanComplement` with the same two integer values. Make use of ternary if-statements in your solution. The output should be “`x is less than y`” if `x` is less than `y`, otherwise “`x is not less than y`”.

5.5.2 Translating Between If and Switch

You can use either if-statements or switch-statements to perform actions conditionally. Which one you use is dependent on the application it is being used for. There are instances when one selection statement is more appropriate in the given situation than another and a translation between the two is required. We will consider the translation from an if-statement to a switch-statement followed by the reverse.

5.5.2.1 From If to Switch

If-statements are more versatile and can evaluate a wide range of conditions, based on a range of variables. However, when you have a single variable being tested against multiple values, a switch-statement often becomes more readable and straightforward.

Consider the following if-else chain that checks an integer variable:

```
int status = getStatus(); // Assume getStatus() returns an int

if (status == 1) {
    cout << "Pending";
} else if (status == 2) {
    cout << "Approved";
}
```



```

} else if (status == 3) {
    cout << "Rejected";
} else {
    cout << "Unknown_status";
}

```

Listing 5.19: If-Else Chain To Display Status

This can be directly translated into a switch-statement as follows:

```

int status = getStatus(); // Assume getStatus() returns an int
switch (status) {
    case 1:
        cout << "Pending";
        break;
    case 2:
        cout << "Approved";
        break;
    case 3:
        cout << "Rejected";
        break;
    default:
        cout << "Unknown_status";
}

```

Listing 5.20: Switch-Statement Equivalent To Display Status

The switch-statement is cleaner and easier to manage, particularly as the number of conditions grows.

Note that the translation from if-statement to switch-statement is only possible when all conditions compare against a single variable/expression.

5.5.2.2 From Switch to If

While switch-statements are useful for checking a single variable against a series of constants, they are limited to equality checks. To handle more complex conditions (eg. ranges) or to involve logical operators, translating to if-statements may be necessary.

Consider a switch-statement that handles different grades given in Listing 5.21.

```

char grade = getGrade(); // Assume getGrade() returns a char

switch (grade) {
    case 'A':
        cout << "Excellent";
        break;
    case 'B':
    case 'C':
    case 'D':
        cout << "You_passed";
        break;
}

```

```

    case 'F':
        cout << "Better_luck_next_time";
        break;
    default:
        cout << "Invalid_grade";
}

```

Listing 5.21: Switch-Statement for Grades

The translation of the given switch-statement to an if-statements needs to take the ranges of distinct values into account. A **grade** of B, C and D all result in an output of “You passed”. This effective is combination of testing of the logical disjunction (the || operator) of the equivalences of **grade** with each of the grade symbols, B, C and D. Note the use of the chaining to create the equivalent statement.

```

if (grade == 'A') {
    cout << "Excellent";
} else if (grade == 'B' || grade == 'C' || grade == 'D') {
    cout << "You_passed";
} else if (grade == 'F') {
    cout << "Better_luck_next_time";
} else {
    cout << "Invalid_grade";
}

```

Listing 5.22: If-Statement Equivalent for Grades

An if-statement can handle a broader range of conditions, such as checking for a range of values or combining multiple conditions, which is not possible with a switch-statement. If you need to check ranges or conditions that involve more than direct equality, you generally use if-statements.

5.5.2.3 Considerations for Translations between Selection Statements

Considerations that need to be taken into account when translating between if-statements and switch-statement.

1. **Use ‘switch’ when:** You have a single variable being compared against multiple distinct constants.
2. **Use ‘if’ when:** Conditions are complex, involve logical operators, or you are checking ranges.

Question 5.20

Translate Listing 5.13 to the equivalent chained else-if format.

5.6 Testing and Debugging

As we progress through the study units, the code we write becomes more complex. The compiler will flag all syntax errors, but other errors such as semantic and logic errors

are not necessarily flagged during the compilation process. To help with testing and debugging, two techniques will be discussed. The first is quick and clumsy, while the second is a much more elegant solution. There is naturally a solution that is better than either of these and that is using a code debugger which falls outside the scope of this module.

5.6.1 Print Statements

The easiest and most rudimentary method is to place `cout` statements before and after the statements where there is a potential for a state change (e.g. the value of a variable changes) or where some condition needs to be met. For example, in the following code the value of `x` needs to be even. Assuming you suspect something is not correct in the calculations before `x` needs to be even, you can place a simple `cout` statement such as `cout << ((x % 2 == 0)) << endl;` to test whether the requirement is true.

The problem with this strategy is that either you need to comment out the `cout` statements used for debugging or delete them before the code can be submitted for use.

5.6.2 Assertion

Assertions in C++ determine whether an expression is true. It is used to check conditions that cannot happen unless there is a bug in the code. `Assert` will terminate the program if the expression becomes false.

To use the assertion debugging aid in your program you need to make use of the `assert` preprocessor macro which is defined in the `cassert` header file.

Below is a basic example of how `assert` can be applied to determine whether the variable `x` is even.

```
#include <cassert>

int main() {
    int x;
    // lines of code to manipulate x
    assert((x % 2 == 0));
    // lines of code that work on the assumption that x is even
    return 0;
}
```

Listing 5.23: Basic Use of the Assert Macro

Unlike print statements that are included in code and need to be removed manually, `assert` can be removed using the `NDEBUG` macro. This is included in the code by adding `#define NDEBUG` before the inclusion of the `cassert` header file.

5.6.3 Using Assertions to Enforce Pre- and Post-conditions

Assertions can be used with great effect to enforce pre- and post-conditions when coding. As they can be switched off, the code can go into production without the statements

producing side-effect with the code. Assert statement can be placed at the beginning of a function to test the pre-condition of the function and just before the function terminated – either before a return statement or the last line of the function if it is a `void` function.

5.7 Document Change Log

Date	Change
17 Apr 2024	Initial preparation and presentation of the document.